Trabalho Prático: Divisão e Conquista

Relatório experimental · Davi Gomes Porfirio Brito e Carlos Trevia

Implementamos em Python, para dois problemas, uma solução de força bruta e uma de divisão e conquista, mais a versão **O(n)** da questão desafio (algoritmo de Kadane) para o subarranjo de soma máxima. Verificamos experimentalmente se o tempo segue as recorrências previstas. Todos os números, tabelas e gráficos deste PDF saem da mesma execução de *benchmark.py* (*resultados/*.csv*); o PDF é gerado por *gerar_relatorio.py*.

1. Metodologia e máquina

Item	Valor
Processador	Intel(R) Core(TM) i7-8565U CPU @ 1.80GHz
Núcleos lógicos / RAM	8 / 7.8 GB
Sistema operacional	Windows-11-10.0.26200-SP0
Python	3.14.7 (CPython)

Tamanhos: 100, 200, 400, 800, 1600, 3200, 6400 (o enunciado pede até 6.400). **10 repetições** por tamanho (mínimo exigido: 5), com uma entrada nova a cada repetição (semente 2026; vetores com inteiros em [-100, 100], pontos com coordenadas inteiras em [-10.000, 10.000]). Mede-se só a chamada do algoritmo, com *perf_counter_ns*, coletor de lixo desligado durante a medição e processo com prioridade alta preso a um núcleo. O Kadane é rápido demais para um único disparo; cada medição dele é a média de várias chamadas seguidas. Como o ambiente é um notebook, há ruído: para o par de pontos com n = 6400, a mesma configuração variou de 41,4 a 150,0 ms entre repetições. Por isso as tabelas trazem a **média** (exigida) e a **mediana** (robusta a picos), e a escolha do limiar usa a mediana.

2. Etapa 1 — Subarranjo de soma máxima

Força bruta. Dois laços aninhados com soma acumulada examinam os n(n+1)/2 intervalos; T(n) = Θ(n²), espaço O(1).

Divisão e conquista. Divide o vetor ao meio, resolve cada metade e calcula, em Θ(n), o melhor intervalo que cruza o meio (melhor sufixo da esquerda + melhor prefixo da direita); a resposta é o maior dos três. Recorrência: **$T(n) = 2T(n/2) + \Theta(n)$**, com T(1) = Θ(1). Teorema mestre: **$a = 2$, $b = 2$, $f(n) = \Theta(n)$**; $n^{\log_b a} = n^{\log_2 2} = n$. Como $f(n) = \Theta(n^{\log_b a})$, vale o caso 2 e **$T(n) = \Theta(n \log n)$**. Espaço auxiliar O(log n) (pilha de recursão).

Versão linear (desafio). Kadane: um único laço mantém a melhor soma de um intervalo que termina na posição atual; **$T(n) = T(n-1) + \Theta(1) = \Theta(n)$** (recorrência por subtração: o teorema mestre não se aplica, pois não há divisão do problema), espaço O(1). Para obedecer à mesma regra de desempate das outras versões, só reinicia o intervalo quando a soma anterior é *negativa* (com soma 0, estender preserva o menor índice inicial) e só troca o melhor quando (−soma, início, fim) é estritamente menor.

Convenções. Índices inclusivos e base 0. Vetor só com negativos devolve o maior elemento, não 0 (intervalo vazio não é permitido). Empate: menor índice inicial e, depois, menor índice final.

3. Etapa 2 — Par de pontos mais próximos

Força bruta. Compara os n(n−1)/2 pares: Θ(n²).

Divisão e conquista. Ordena os pontos por x e por y uma única vez (Θ(n log n)). Em cada chamada: divide por uma linha vertical na mediana de x (Θ(n)), particiona a lista ordenada por y entre as metades (Θ(n)), resolve as duas metades obtendo $\delta = \min(\delta_E, \delta_D)$ e examina a faixa de largura 2δ em torno da linha, comparando cada ponto com no máximo 7 sucessores na ordem de y (Θ(n)). Recorrência: **$T(n) = 2T(n/2) + \Theta(n)$**; **$a = 2$, $b = 2$, $f(n) = \Theta(n)$**, caso 2 do teorema mestre: Θ(n log n). Somada à ordenação inicial Θ(n log n), o total continua **$\Theta(n \log n)$**. Subproblemas com até *limiar* pontos (padrão 8, Seção 7) usam força bruta.

Por que bastam 7 comparações por ponto na faixa.

Seja p um ponto da faixa e considere só os q com y_q entre y_p e $y_p + \delta$: qualquer outro q está a mais de δ de p na vertical e não melhora a resposta. Esses pontos ficam no retângulo 2δ × δ de $x_m - \delta$ a $x_m + \delta$, acima de p. Dividimos o retângulo em **8 quadrados de lado δ/2** (4 colunas × 2 linhas); a linha mediana passa entre a 2ª e a 3ª coluna, então cada quadrado fica inteiro de um lado. Dois pontos do *mesmo lado* distam pelo menos δ (δ é o mínimo de cada metade), mas dois pontos no mesmo quadrado distam no máximo $(\delta/2)\sqrt{2} = \delta/\sqrt{2} < \delta$. Logo cada quadrado tem no máximo 1 ponto e o retângulo tem no máximo 8, contando o próprio p: restam **no máximo 7 sucessores** a comparar. Pontos sobre a linha pertencem à metade em que foram colocados e a contagem vale por metade (no máximo 4 pontos em cada retângulo δ × δ), então o limite de 8 se mantém; os casos *par_grade_6x6* e *par_mesmo_x_21* exercitam esse cenário.

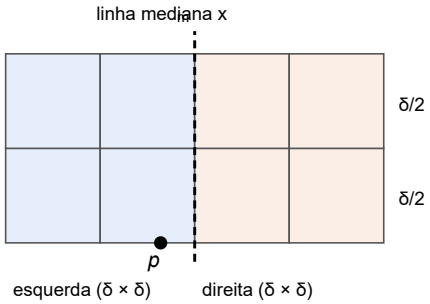


Figura — Retângulo 2δ × δ acima de p dividido em 8 quadrados de lado δ/2; a linha tracejada é a mediana em x.

4. Resultados — Subarranjo de soma máxima

Tabela 1 — Soma máxima: tempo (ms), média e mediana de 10 repetições.

n	Bruta média	Bruta mediana	D&C média	D&C mediana	Kadane média	Kadane mediana	Bruta / D&C (médias)
100	5,568	5,048	0,509	0,458	0,0374	0,0316	10,9×
200	20,435	20,638	1,145	0,949	0,0857	0,0660	17,8×
400	88,169	88,772	1,981	1,886	0,1384	0,1278	44,5×
800	249,476	232,533	3,289	2,958	0,2002	0,1957	75,8×
1600	1.128,017	970,308	7,866	5,939	0,4563	0,3891	143,4×
3200	5.225,191	4.545,382	17,476	14,698	1,2510	0,9810	299,0×
6400	21.401,040	20.762,965	30,607	26,425	1,8035	1,5930	699,2×

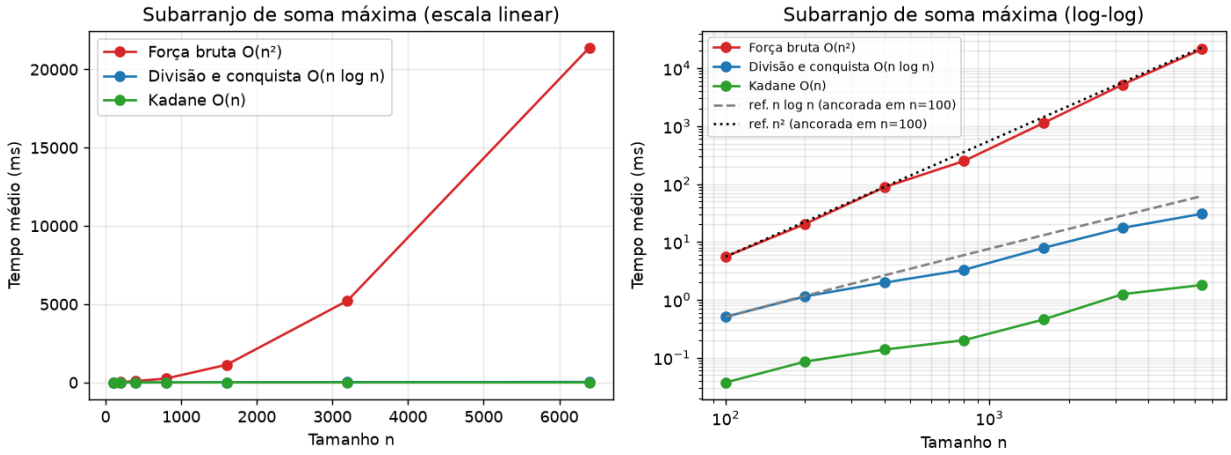


Figura 1 — Soma máxima (médias). Esquerda: escala linear. Direita: log-log com referências n log n e n² ancoradas em n = 100; o Kadane está abaixo das demais.

5. Resultados — Par de pontos mais próximos

Tabela 2 — Par mais próximo: tempo (ms), média e mediana.

n	Bruto média	Bruto mediana	D&C média	D&C mediana	Bruto / D&C
100	4,594	4,212	0,720	0,612	6,4×
200	18,288	18,178	1,574	1,432	11,6×
400	74,072	72,523	3,089	2,768	24,0×
800	221,147	200,036	5,610	4,668	39,4×
1600	1.023,906	792,912	12,362	9,784	82,8×
3200	4.371,853	4.214,667	29,415	25,358	148,6×
6400	16.656,561	15.683,755	70,567	57,143	236,0×

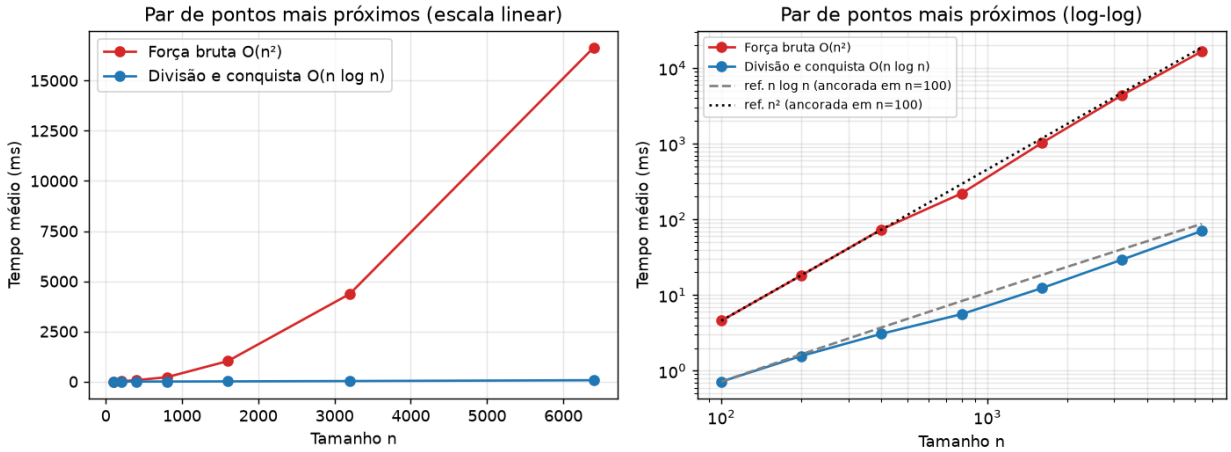


Figura 2 — Par mais próximo (médias), mesma convenção da Figura 1.

6. Discussão: observado × previsto

Tabela 3 — Expoente α de $T \approx c \cdot n^{\alpha}$ (regressão log-log nos 7 tamanhos) e razão média ao dobrar n.

Algoritmo	Expoente (média)	Expoente (mediana)	Esperado	Razão T(2n)/T(n)	Esperada
Soma — força bruta	1,98	1,97	2	4,01	4,00
Soma — divisão e conquista	0,99	0,97	≈ 1,1 a 1,2	2,00	2,2
Soma — Kadane O(n)	0,94	0,94	1	1,97	2,00
Par — força bruta	1,97	1,96	2	3,95	4,00
Par — divisão e conquista	1,08	1,06	≈ 1,1 a 1,2	2,16	2,2

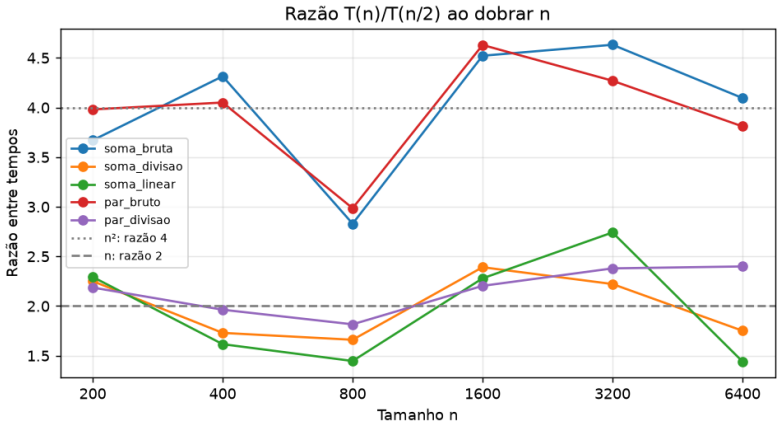


Figura 3 — Razão T(n)/T(n/2) em cada dobra de n.

n² contra n log n. As duas forças brutas têm expoente 1,98 e 1,97 e razão média 4,01 e 3,95: quadrupla ao dobrar n, como previsto para Θ(n²). As versões de divisão e conquista têm expoentes 0,99 (soma) e 1,08 (par) e razões em torno de 2: crescimento quase linear, compatível com Θ(n log n), cuja razão esperada vai de 2,30 (100 → 200) a 2,17 (3.200 → 6.400). **Limitação honesta:** em apenas seis dobras de n, com ruído de notebook e sobrecarga do interpretador, os dados *não distinguem* n log n de n; o que eles mostram com clareza é o contraste com n² (razão ≈ 2 contra ≈ 4). O Kadane tem expoente 0,94 e razão 1,97, linear como previsto, e é cerca de 15× mais rápido que a divisão e conquista em todos os tamanhos (diferença só de fator constante: recursão, fatiamento e comparações de

tuplas). A vantagem da técnica sobre a força bruta cresce com n: 10,9× em n = 100 e 699× em n = 6400 na soma; 6,4× e 236× no par. Em n = 6400 a força bruta leva 21,4 s (soma) e 16,7 s (par), contra 31 ms e 71 ms.

Quem vence em cada tamanho e o ponto de cruzamento

Tabela 4 — Tamanhos pequenos: tempo médio (ms). A divisão e conquista usa recursão pura (subproblemas de até 3 pontos vão à força bruta).

n	Soma: bruta	Soma: D&C	Soma: Kadane	Par: bruto	Par: D&C
4	0,0106	0,0127	0,0016	0,0072	0,0153
8	0,0421	0,0331	0,0033	0,0332	0,0403
16	0,1351	0,0603	0,0045	0,1048	0,0856
32	0,5095	0,1405	0,0113	0,4347	0,1793
64	2,3810	0,3113	0,0251	2,0272	0,4606
128	7,5590	0,5334	0,0440	6,5680	0,9084
256	29,3899	1,1236	0,0768	25,4583	1,7602

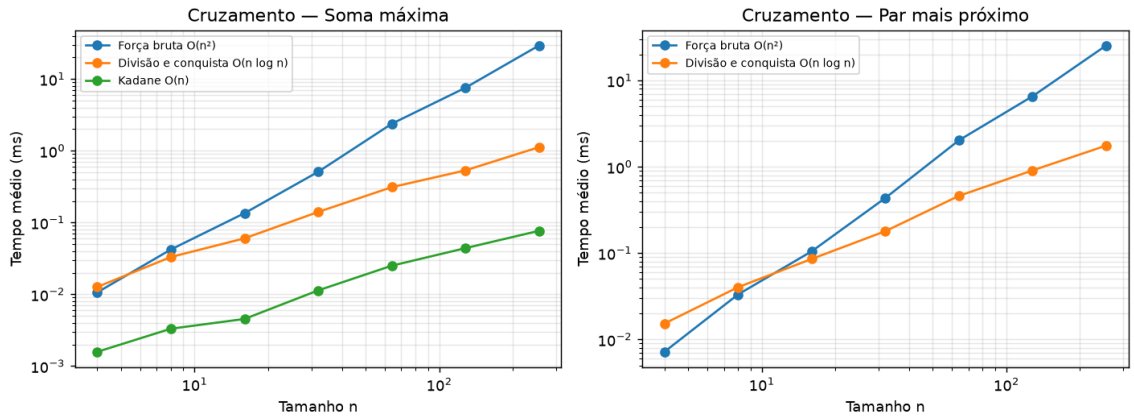


Figura 4 — Cruzamento entre força bruta e divisão e conquista (log-log).

Para tamanhos pequenos a **força bruta vence**: na soma, até n = 4; no par, até n = 8. O cruzamento ocorre entre n = 4 e 8 na soma e entre n = 8 e 16 no par. Abaixo disso o custo é dominado por constantes (chamadas recursivas, fatiamento, ordenações, conjuntos de identificadores no par) enquanto n² ainda é pequeno. Acima, n² cresce mais depressa que n log n e a diferença só aumenta. O Kadane vence os dois desde o menor tamanho medido, porque faz uma passada simples sem recursão. O cruzamento do par (≈ 8 a 16 pontos) é a base prática do limiar da próxima seção.

7. Limiar da Etapa 2

Tabela 5 — Varredura do limiar: mesma entrada para todos os limiares em cada repetição. Pontuação = mediana / melhor mediana entre os limiares, média entre os tamanhos (1,000 = melhor).

Limiar	mediana n=1600 (ms)	mediana n=3200 (ms)	mediana n=6400 (ms)	Pontuação
2	9,71	23,70	115,45	1,116
4	9,40	21,75	98,46	1,016
8	9,20	21,56	96,84	1,000 ← escolhido
16	11,04	27,89	112,15	1,217
32	15,89	35,47	183,56	1,756
64	27,90	64,21	319,81	3,105

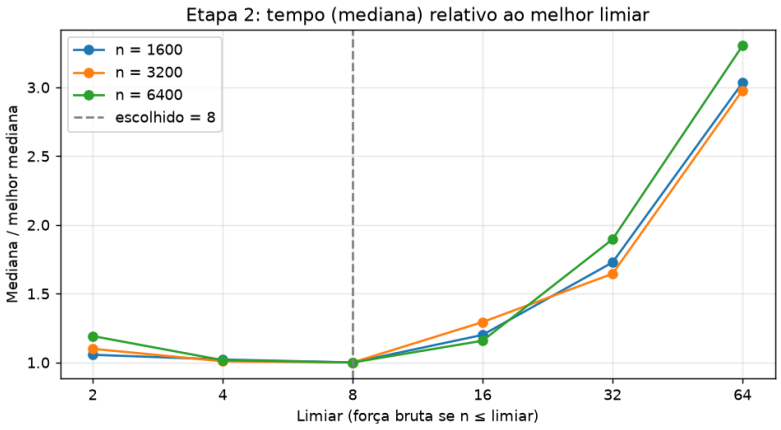


Figura 5 — Tempo relativo ao melhor limiar, por tamanho de entrada.

Escolha: limiar 8. É o de menor pontuação nos três tamanhos testados. Os limiares 4 e 8 empatam (diferença de 1,6%, dentro do ruído); o 2 é 12% pior e o 16, valor que usávamos antes sem calibrar, é 22% pior, o que mostra que não era ótimo. A partir de 32 o custo sobe depressa (1,76× e 3,10× o melhor). A forma da curva tem explicação: cada folha de L pontos custa $\approx c_1 \cdot L^2$ e há n/L folhas, logo as folhas somam $\approx c_1 \cdot n \cdot L$ (cresce com L); já a sobrecarga da recursão ($\approx c_2$ por chamada, n/L chamadas) diminui com L. O mínimo fica onde as duas se equilibram, perto de $L^* \approx \sqrt{(c_2/c_1)}$, e a curva é plana ao redor dele. Isso concorda com a Tabela 4, em que a força bruta ainda vence até ≈ 8 pontos. Os limiares 2 e 3 são equivalentes na implementação (com n ≤ 3 dividir geraria uma metade de 1 ponto, sem par). O valor ótimo depende da máquina e do interpretador; o *benchmark.py* refaz a escolha e avisa se o padrão de *algoritmos.py* divergir.

8. Testes e casos de borda

tests/casos_teste.json reúne 30 casos (19 de soma máxima, 11 de par de pontos) com entrada, saída esperada e justificativa; cada saída foi derivada à mão e conferida com a força bruta. Eles rodam em todas as implementações e, no par, em todos os limiares (2 a 64), e também pelo CLI (*main.py*, entrada por stdin). Além disso há testes aleatórios com semente fixa que comparam as versões; na soma, valores em [−2, 2] forçam muitos empates.

Caso de borda	Por que importa
Só negativos; negativos iguais	A resposta é o maior elemento, não 0; ensina que o intervalo vazio é proibido e que o desempate vale entre elementos iguais.
n = 1 (positivo, negativo, zero)	Caso base da recursão e do laço; índices (0, 0).
Empates: simples, vários intervalos, alternado, cruzando o meio	Verificam a regra menor início → menor fim e a combinação esquerda/direita/cruzado (ex.: [2,2,−4,4] → (4,0,1)).
Todos zeros; zeros entre negativos; zero antes do máximo	Todos os intervalos empatam; o Kadane precisa estender com soma 0 para preservar o menor início.
Pontos coincidentes; 45 pontos em 3 posições	Distância 0, vários pares empatados e faixa lotada de pontos iguais.
Colineares: mesmo y, mesmo x, diagonal	Faixa degenerada (todos na faixa ou todos em uma linha) e empate de raiz de 2 na diagonal.
Grade 6×6; 21 pontos com mesmo x	Dezenas de pares empatados e linha mediana cheia de pontos (valida o limite de 7 vizinhos).
Par que cruza a mediana (n = 20 > limiar)	Só a combinação na faixa encontra o melhor par; as metades isoladas não.
Limiares 2, 3, 4, 8, 16, 32, 64	O resultado não pode depender do limiar; achou um defeito real com limiar 2 (subproblema de 1 ponto), já corrigido.

9. Conclusões, limitações e reprodução

As recorrências $T(n) = 2T(n/2) + \Theta(n)$ ($a = 2$, $b = 2$, $f(n) = \Theta(n)$, caso 2) valem nos dois problemas e a força bruta quadrática é muito superada a partir de algumas dezenas de elementos; o Kadane é o melhor para a soma máxima. Limitações: medições em um único notebook, com ruído; faixa de apenas 64× em n, que separa n^2 de n log n mas não n log n de n; o limiar ótimo depende de máquina e linguagem. Reprodução: *python -m unittest discover -s tests*, *python benchmark.py* e *python gerar_relatorio.py*.